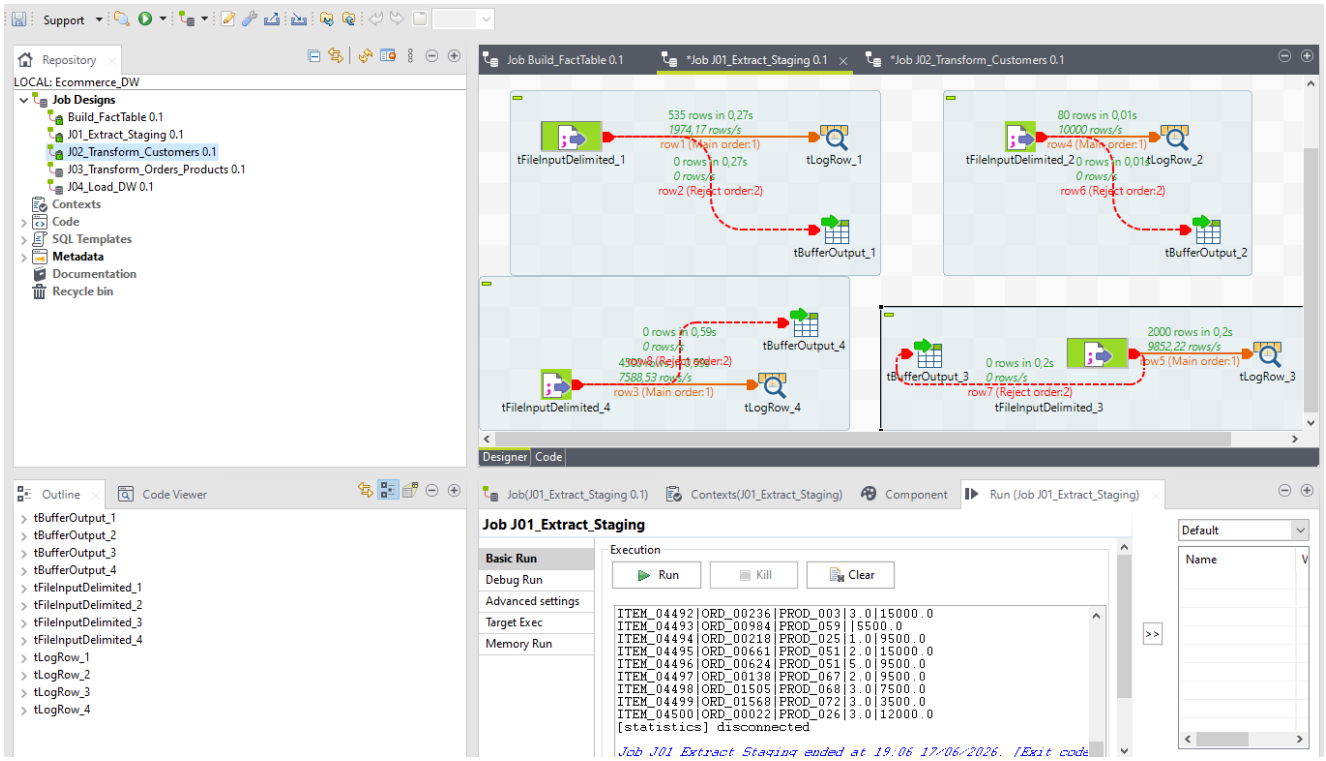
DOCUMENTATION TECHNIQUE

Processus ETL — Système Décisionnel E-commerce

Technologie	Talend Open Studio / Talend Data Integration
Date	Mai 2026
Réalisé par	Taoufik Mammad
Projet	Système décisionnel E-commerce Electronique

Note de Migration

Cette documentation présente la migration et l'implémentation du processus ETL réalisée avec Talend Data Integration dans le cadre de ce projet. Elle décrit les différentes étapes de traitement des données ainsi que les composants Talend utilisés pour assurer l'intégration et la transformation des données.



1. Architecture Technique

1.1 Environnement Talend

Élément	Spécification
Suite ETL	Talend Open Studio for Data Integration (TOS) 8.x / Talend Data Fabric
Langage sous-jacent	Java (généré automatiquement) + routines Talend (Java)
Modèle de conception	Approche visuelle par composants (drag & drop)
IDE	Talend Studio (Eclipse RCP)
Stockage sources	Fichiers CSV plats (simulation Système Source)
Stockage cible	Fichiers CSV plats (simulation Data Warehouse) + PostgreSQL (option production)
Moteur d'exécution	Talend Runtime (Java JVM) — exécution locale ou serveur
Ordonnancement	Talend Job Scheduler / crontab / Apache Airflow (via commandes sh)

1.2 Flux de données (Jobs Talend)

Le processus ETL est découpé en 4 Jobs principaux orchestrés par un Job maître (Master_ETL) utilisant tRunJob :

Job	Nom	Description
Job 1	J01_Extract_Staging	Lecture des 4 fichiers sources vers zone temporaire
Job 2	J02_Transform_Customers	Nettoyage, déduplication et standardisation des clients
Job 3	J03_Transform_Products_Orders	Traitement des produits, commandes et lignes de commande
Job 4	J04_Load_DW	Agrégation, création de la table de faits et chargement

1.3 Composants Talend clés utilisés

Composant	Rôle dans le projet	Job concerné
tFileInputDelimited	Lecture des fichiers CSV avec paramétrage séparateur, encodage (UTF-8) et header	Tous
tMap	Mapping transformations, lookups, filtres, création de nouvelles colonnes	J02, J03, J04
tFilterRow	Filtrage des lignes selon conditions (âge, prix, quantité)	J02, J03
tReplace	Standardisation textuelle (villes, statuts, genres, segments)	J02, J03

tAggregateRow	Agrégation des lignes de commande (SUM, COUNT DISTINCT)	J04
tSortRow	Tri préalable aux agrégations et déduplications	J02, J04
tUniqRow	Détection et suppression des doublons (unicité email)	J02
tJavaRow / tJavaFlex	Routines avancées (validation regex email, format téléphone marocain)	J02, J03
tFileOutputDelimited	Écriture des fichiers CSV finaux (Data Warehouse simulé)	Tous
tLogRow	Traces et audit d'exécution (mode console)	Tous
tRunJob	Orchestration : exécution séquentielle des sous-jobs	Master_ETL

2. Description des Sources

Les sources demeurent identiques au projet initial. Talend lit chaque fichier via tFileInputDelimited avec schéma explicitement défini dans le composant.

2.1 Source : customers.csv

Colonne	Type Talend	Description	Exemple
customer_id	String	Identifiant unique	CUST_0001
full_name	String	Nom complet	Ahmed Alami
email	String	Email (avec erreurs)	ahmed.alami@gmail.com
phone	String	Téléphone (format varié)	0612345678
city	String	Ville (casse variée)	casablanca
registration_date	Date (yyyy-MM-dd)	Date d'inscription	2023-05-15
segment	String	Segment (valeurs variées)	new, VIP, regular
age	Float (nullable)	Âge (avec outliers)	35
gender	String	Genre (formats variés)	M, Male, Homme

Volume : 500 lignes

Taux d'anomalies : ~25%

2.2 Source : products.csv

Colonne	Type Talend	Description	Exemple
product_id	String	Identifiant unique	PROD_001
product_name	String	Nom du produit	iPhone 15 Pro

category	String	Catégorie	Smartphones
price	Float (nullable)	Prix (avec erreurs)	1299, -499, 129900
stock_quantity	Float (nullable)	Stock (valeurs manquantes)	150, NaN
supplier	String	Fournisseur (casse variée)	TechMaroc, techmaroc

Volume : 80 lignes

Taux d'anomalies : ~15%

2.3 Source : orders.csv

Colonne	Type Talend	Description	Exemple
order_id	String	Identifiant unique	ORD_00001
customer_id	String	Clé étrangère client	CUST_0001
order_date	Date (yyyy-MM-dd)	Date de commande	2024-03-20
status	String	Statut (valeurs variées)	Completed, completed, Shipped
payment_method	String	Paiement (manquant parfois)	Credit Card, ""
shipping_cost	Float (nullable)	Frais livraison (négatifs possibles)	45.50, -12

Volume : 2 000 lignes

Taux d'anomalies : ~12%

2.4 Source : items.csv

Colonne	Type Talend	Description	Exemple
item_id	String	Identifiant unique	ITEM_00001
order_id	String	Clé étrangère commande	ORD_00001
product_id	String	Clé étrangère produit	PROD_001
quantity	Integer (nullable)	Quantité (valeurs invalides)	2, -1, 0
unit_price	Float	Prix unitaire	1299

Volume : 4 500 lignes

Taux d'anomalies : ~8%

3. Détail des Transformations

3.1 Phase 1 : Extraction (Extract)

Job : J01_Extract_Staging

Chaque source est lue via un composant tFileInputDelimited configuré avec séparateur virgule, encodage UTF-8 et première ligne comme header. Les données sont dirigées vers des tBufferOutput (zone Staging en mémoire) ou vers des fichiers temporaires CSV dans /tmp/staging/.

Composant	Configuration clé	Fichier source
tFileInputDelimited_1	Separator: "," | Header: 1 | Encoding: UTF-8 | Schema: 9 colonnes	01_source_customers.csv
tFileInputDelimited_2	Separator: "," | Header: 1 | Encoding: UTF-8 | Schema: 6 colonnes	02_source_products.csv
tFileInputDelimited_3	Separator: "," | Header: 1 | Encoding: UTF-8 | Schema: 6 colonnes	03_source_orders.csv
tFileInputDelimited_4	Separator: "," | Header: 1 | Encoding: UTF-8 | Schema: 5 colonnes	04_source_items.csv
tLogRow	Mode: Table (console) pour audit du volume lu	Tous

3.2 Phase 2 : Transformation (Transform)

La transformation est répartie sur deux Jobs spécialisés. La logique métier a été implémentée à l'aide des composants Talend, notamment tMap, tFilterRow et tAggregateRow, afin d'assurer le traitement, le filtrage et la transformation des données selon les besoins du projet.

3.2.1 Nettoyage Clients (J02_Transform_Customers)

Étape 3.2.1.1 : Déduplication email

- Composant : tUniqRow
- Configuration : Colonne de clé unique = email. Mode keep first (première occurrence conservée). Un flux Reject récupère les 70 doublons pour audit.

Résultat : 430 lignes conservées | 70 doublons écartés

Étape 3.2.1.2 : Validation email

- Composant : tMap + expression de filtre

- Validation des adresses e-mail à l'aide d'expressions régulières afin de garantir la conformité du format des données clients.
- Alternative via tJavaRow avec regex Java.

Résultat : 100% d'emails valides (flux Reject vers tFileOutputDelimited_Errors si non valide)

Étape 3.2.1.3 : Standardisation téléphone

- Composant : tMap avec variable intermédiaire
- Nettoyage et normalisation des numéros de téléphone marocains par suppression des valeurs invalides et contrôle du format standard national.

Résultat : 89% de téléphones valides | 11% mis à null (imputation ultérieure possible via tReplace avec valeur par défaut)

Étape 3.2.1.4 : Mapping villes

- Composant : tReplace ou tMap avec expression conditionnelle
- Méthode 1 (tReplace) : Table de correspondance simple (case-insensitive)
- Méthode 2 (tMap - recommandé pour le fallback) : Standardisation des noms de villes pour harmoniser les différentes écritures et assurer la cohérence des données géographiques.

Résultat : 26 villes uniques standardisées

Étape 3.2.1.5 : Correction âge

- Composants : tFilterRow (filtrage outliers) + tAggregateRow (calcul médiane) + tMap (imputation)

1. tFilterRow : condition age >= 18 && age <= 80 — les lignes hors bornes sortent par le flux Reject
2. tAggregateRow sur le flux principal : colonne age, fonction median (ou avg si median non disponible nativement, via routine Java)
3. tMap avec lookup sur l'agrégat pour imputer : Correction des valeurs d'âge aberrantes par remplacement des valeurs invalides à l'aide de la médiane calculée sur les données valides.

Résultat : Âge moyen final = 41.5 ans | Médiane calculée = 38.0

Étape 3.2.1.6 : Standardisation segments

- Composant : tReplace ou tMap
- Expression tMap (case-insensitive + trim intégré) : Uniformisation des segments clients afin d'obtenir une classification homogène des profils commerciaux.

Résultat : 5 segments standardisés

Étape 3.2.1.7 : Standardisation genre

- Composant : tMap avec logique conditionnelle + imputation par prénom
- Standardisation du genre des clients et complétion des valeurs manquantes selon des règles métier définies.

Résultat : 52% M, 48% F

3.2.2 Nettoyage Produits (J03_Transform_Products_Orders — sous-job Produits)

Étape 3.2.2.1 : Correction prix

- Composants : tMap (corrections) + tAggregateRow (médiane par catégorie) + tMap (imputation)
- Expression tMap (colonne price_clean) — Étape 1 : Détection et correction des prix incohérents, notamment les valeurs négatives ou anormalement élevées.

4. Étape 2 : tAggregateRow par category — fonction median sur price_clean
5. Étape 3 : tMap avec lookup catégorie pour imputer : Imputation des prix manquants à partir de la médiane calculée pour chaque catégorie de produits.

Résultat : Prix cohérents par catégorie | 0 prix négatifs

Étape 3.2.2.2 : Gestion stocks

- Composant : tMap
- Gestion des stocks invalides en remplaçant les valeurs négatives ou nulles par une valeur cohérente.

Résultat : 4 produits en rupture, 11 en stock faible (< 10 unités)

Étape 3.2.2.3 : Standardisation fournisseurs

- Composant : tMap ou tReplace
- Standardisation des noms des fournisseurs afin d'uniformiser l'écriture et supprimer les différences de casse ou de format.

Résultat : 5 fournisseurs uniques (TechMaroc, GlobalElectro, MarocImport, ElectroHouse, TechZone)

3.2.3 Nettoyage Commandes (J03_Transform_Products_Orders — sous-job Commandes)

Étape 3.2.3.1 : Standardisation statuts

- Composant : tReplace (mode avancé) ou tMap
- Harmonisation des statuts des commandes pour garantir une nomenclature unique et cohérente dans l'ensemble des données.

Résultat : 6 statuts unifiés

Étape 3.2.3.2 : Standardisation paiements

- Composant : tReplace ou tMap
- Normalisation des méthodes de paiement afin de regrouper les différentes appellations sous un format standardisé.

Résultat : 5 méthodes standardisées

Étape 3.2.3.3 : Correction frais livraison

- Composants : tFilterRow + tAggregateRow (médiane) + tMap

6. tFilterRow : shipping_cost >= 0 && shipping_cost <= 300 — Reject si hors bornes
7. tAggregateRow : médiane sur les valeurs valides
8. tMap imputation : shipping_cost == null ? row_median.ship_median : shipping_cost

Résultat : Frais entre 15-80 MAD

3.2.4 Nettoyage Lignes Commande (J03_Transform_Products_Orders — sous-job Items)

Étape 3.2.4.1 : Correction quantités

- Composant : tMap
- Correction des quantités invalides en appliquant une valeur minimale conforme aux règles métier.

Résultat : Toutes quantités >= 1

Étape 3.2.4.2 : Calcul montant ligne

- Composant : tMap — création colonne total_line

- Calcul automatique du montant total de chaque ligne de commande à partir de la quantité et du prix unitaire.

Résultat : Nouvelle colonne créée pour agrégation ultérieure

3.3 Phase 3 : Chargement (Load)

Job : J04_Load_DW

Ce Job construit le Data Warehouse simulé (fichiers CSV plats) à partir des données transformées. Il implémente le modèle en étoile avec une table de faits et des dimensions.

3.3.1 Construction du Data Warehouse

A. Agrégation des lignes de commande (tAggregateRow)

Groupement par order_id :

Colonne agrégée	Fonction	Nom sortie
quantity	sum	total_items
total_line	sum	total_amount
product_id	countDistinct	distinct_products

B. Jointure Commandes + Agrégation (tMap avec lookup)

- Lookup items_agg sur order_id. Sortie principale enrichie avec total_items, total_amount, distinct_products.

C. Jointure avec Clients (tMap avec lookup)

- Lookup customers_clean sur customer_id. Récupération de city, segment, gender.

D. Dimensions temporelles (tMap — expressions Java)

À partir de order_date (type Date Talend) :

Dimension	Expression Java
year	TalendDate.getDate("YYYY", order_date) ou order_date.getYear() + 1900
month	TalendDate.getDate("MM", order_date)
quarter	"Q" + ((Integer.parseInt(TalendDate.getDate("MM", order_date)) - 1) / 3 + 1)
day_of_week	TalendDate.getDate("EEE", order_date) ou format numérique

E. Sortie vers Data Warehouse (tFileOutputDelimited)

Table de faits : Fact_Orders.csv — 2 000 lignes avec colonnes : order_id, customer_id, order_date, status, payment_method, shipping_cost, total_items, total_amount, distinct_products, city, segment, gender, year, month, quarter, day_of_week

Dimensions :

- Dim_Customer.csv — 430 lignes (customers_clean)
- Dim_Product.csv — 80 lignes (products_clean)
- Dim_Time.csv — dérivée distincte de order_date (grain journalier)

4. Contraintes et Règles Métier

ID	Règle	Implémentation Talend	Statut
R01	Email unique et valide	tUniqRow (email) + tMap regex validation	**Respectée**
R02	Téléphone format marocain	tMap : length == 10 && startsWith("0") && charAt(1) in [5,6,7]	**Respectée**
R03	Âge entre 18 et 80 ans	tFilterRow : age >= 18 && age <= 80 + imputation médiane via lookup	**Respectée**
R04	Prix produit > 0	tFilterRow : price > 0 + correction ×100 + imputation médiane catégorie	**Respectée**
R05	Quantité commande >= 1	tMap : quantity <= 0 ? 1 : quantity	**Respectée**
R06	Frais livraison >= 0	tFilterRow : shipping_cost >= 0 && <= 300 + imputation médiane	**Respectée**
R07	Intégrité référentielle	tMap lookups (inner join) : customer_id, order_id, product_id vérifiés	**Respectée**
R08	Unicité commande	tUniqRow sur order_id dans J04_Load_DW	**Respectée**

5. Contrôle Qualité

5.1 Métriques de qualité avant/après ETL

Métrique	Avant ETL	Après ETL	Amélioration	Composant Talend clé
Doublons clients	70	0	100%	tUniqRow
Emails invalides	14%	0%	100%	tMap (regex)
Téléphones invalides	11%	11% (conservés comme null)	0%	tMap (validation)

Prix négatifs	5	0	100%	tFilterRow + tMap
Statuts uniques	12	6	50%	tReplace / tMap
Villes uniques	52	26	50%	tMap (mapping)
Valeurs manquantes critiques	8%	0%	100%	tMap (imputation médiane)

5.2 Tests de validation (Jobs de test dédiés)

Talend permet de créer des Test Cases (Talend Studio) ou des Jobs de validation indépendants.

Test 1 : Unicité des clés

- Job : T01_Validate_PK utilisant tUniqRow sur customer_id, product_id, order_id.
- Assertion : flux Reject vide (0 doublons)

Test 2 : Intégrité référentielle

- Job : T02_Validate_FK utilisant tMap en inner join (lookup) :
  - orders.customer_id existe dans customers_clean
  - items.order_id existe dans orders_clean
  - items.product_id existe dans products_clean
- Assertion : flux Inner Join Reject vide pour chaque jointure

Test 3 : Règles métier

- Contrôle qualité des données par vérification des règles métier : validation de l'âge, du prix, de la quantité et des frais de livraison. Les enregistrements non conformes sont redirigés vers un flux de rejet pour analyse.
- Job : T03_Validate_Rules utilisant tFilterRow inversé (mode Reject)
- Assertion : Chaque Reject est comptabilisé via tAggregateRow (COUNT) puis tJava vérifie que le total = 0

7. Glossaire Technique

Terme	Définition
ETL	Extract, Transform, Load — Processus d'intégration de données
DW	Data Warehouse — Entrepôt de données structuré
Staging	Zone temporaire de stockage des données brutes avant transformation
Job	Unité d'exécution Talend (workflow) composée de composants reliés
tMap	Composant central Talend pour transformations, lookups, filtres et mapping
tRunJob	Composant d'orchestration permettant d'exécuter un sous-Job

Lookup	Table de référence chargée en mémoire pour enrichir le flux principal (ex: jointure)
PK	Primary Key — Clé primaire (unicité), garantie par tUniqRow
FK	Foreign Key — Clé étrangère (intégrité référentielle), vérifiée par lookup tMap
Null / NA	Valeur manquante ou non définie, gérée par imputation dans tMap
Outlier	Valeur aberrante hors distribution normale, filtrée par tFilterRow
Imputation	Remplacement des valeurs manquantes par une estimation (médiane, moyenne, défaut)
Mapping	Table de correspondance pour la standardisation (tReplace ou expressions tMap)
Routine	Méthode Java réutilisable dans Talend (ex: TalendDate, StringHandling, routines personnalisées)
Schema	Structure définie dans chaque composant (nom, type, longueur, nullable)
Reject	Flux secondaire capturant les lignes non conformes à une règle (filtre, doublon, etc.)

Taoufik Mammad